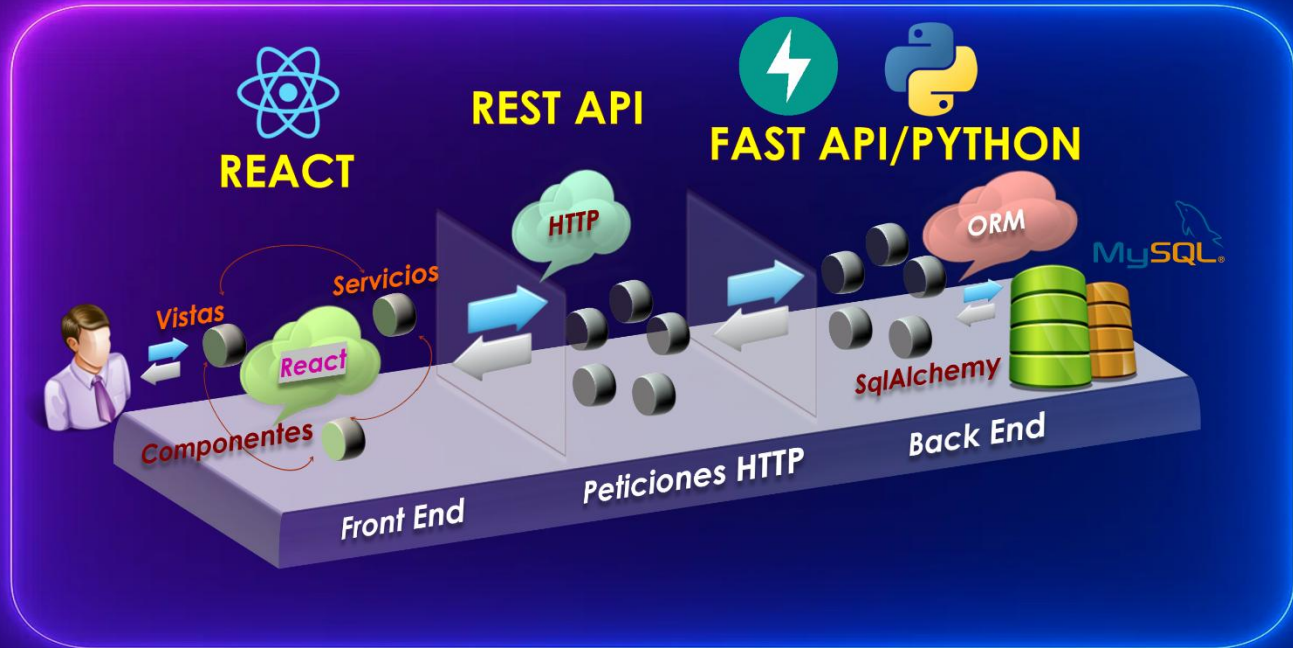


Aplicación Full Stack



✓ FASE 1 — CONFIGURACIÓN INICIAL DEL PROYECTO

Objetivo de la fase:

Dejar listo el proyecto base de React + Vite en la carpeta `C:\Cursos\React\biblio-react`, con axios instalado y la aplicación ejecutándose correctamente en el navegador.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
C:\
├── Cursos\
│   ├── React\
│   │   └── biblio-react\ (NUEVO - proyecto Vite generado)
```

En esta fase solo creamos la carpeta del proyecto y dejamos que Vite genere su estructura interna. Aún **no modificamos** ningún archivo específico del proyecto.

1 Crear la carpeta base para proyectos React

1. Abre el Explorador de archivos de Windows.
2. Crea la carpeta:

3. `C:\Cursos\React\`
 4. Dentro de esa carpeta se creará el proyecto `biblio-react` con Vite.
-

2 Abrir la carpeta en Visual Studio Code

1. Abre **Visual Studio Code**.
 2. Ve al menú:
File → **Open Folder...** (Archivo → Abrir carpeta...)
 3. Selecciona:
 4. `C:\Cursos\React\`
 5. Haz clic en **Select Folder / Seleccionar carpeta**.
-

3 Crear el proyecto con Vite (plantilla React + JS)

En Visual Studio Code:

1. Abre la **terminal integrada**:
Menú: **View** → **Terminal** (Ver → Terminal).
2. En la terminal, ejecuta:
3. `npm create vite@latest biblio-react -- --template react`
4. Responde a las preguntas (pueden variar ligeramente, pero la idea es esta):
 - **Ok to proceed? (y)**
Escribe: `y` y presiona Enter.
 - **Select a framework:**
Selecciona: `React`.
 - **Select a variant:**
Selecciona: `JavaScript`.
 - **Use rolldown-vite (Experimental)?**
Selecciona: `No`.
 - **Install with npm and start now?**
Selecciona: `Yes`
(esto hará el `npm install` automáticamente dentro del proyecto `biblio-react`).

Al finalizar, se habrá creado la carpeta `biblio-react` con la estructura estándar de un proyecto Vite + React.

4 Abrir la carpeta del proyecto en VS Code

Ahora sí vamos a trabajar directamente sobre el proyecto.

1. En Visual Studio Code ve a:
File → Open Folder...
2. Selecciona la carpeta:
3. C:\Cursos\React\biblio-react\
4. Haz clic en **Select Folder**.

A partir de esta fase, todo lo que hagamos será dentro de `biblio-react`.

5 Instalar axios (aún NO react-router-dom)

En la terminal integrada de VS Code, asegurándote de estar en:

```
C:\Cursos\React\biblio-react\
```

Ejecuta:

```
npm install axios
```

Aquí **solo** instalamos `axios`.

`react-router-dom` se instalará **más adelante** (según el plan, hasta la Fase 7).

6 Ejecutar la aplicación inicial

1. En la terminal, dentro de `biblio-react`, ejecuta:
2. `npm run dev`
3. La terminal te mostrará una URL similar a:
4. `http://localhost:5173/`
5. Abre esa URL en tu navegador.
6. Verifica que se vea la pantalla inicial de Vite + React (la vista por defecto que genera el proyecto).

En esta fase **no modificamos todavía ningún archivo** del código fuente; solo confirmamos que el entorno está funcionando correctamente.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 2)**.

Dijiste:

listo vamos fase 2

ChatGPT dijo:

✓ FASE 2 — AGREGAR BOOTSTRAP EN MODO OSCURO + TÍTULO INICIAL

Objetivo de la fase:

Integrar **Bootstrap 5 en modo oscuro**, habilitar **Bootstrap Icons**, eliminar los estilos CSS por defecto generados por Vite, y mostrar un **título centrado horizontalmente** (no vertical) con un **icono de biblioteca**.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\
├── index.html                (MODIFICADO)
├── src\
│   ├── main.jsx             (MODIFICADO)
│   ├── App.jsx              (MODIFICADO)
│   ├── index.css            (ELIMINADO)
│   └── App.css               (ELIMINADO)
```

Recordatorio: En archivos nuevos no agregamos marcas.

En archivos existentes mostramos // [MODIFICADO] solo en líneas cambiadas.

1 Modificar index.html para activar Bootstrap Dark + Icons

Ruta:

biblio-react/index.html

Descripción:

Se agrega Bootstrap 5 vía CDN, Bootstrap Icons y el modo oscuro con `data-bs-theme="dark"`.

Archivo completo (MODIFICADO):

```
<!DOCTYPE html>
<html lang="en" data-bs-theme="dark"> <!-- [MODIFICADO] -->
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Biblioteca Personal</title>

    <!-- Bootstrap 5 Dark Mode -->
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
    />

    <!-- Bootstrap Icons -->
    <link
```

```
    rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.css"
  />
</head>
<body>
  <div id="root"></div>

  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  ></script>
</body>
</html>
```

2 Modificar main.jsx para eliminar importación de CSS por defecto

Ruta:

biblio-react/src/main.jsx

Descripción:

Quitamos la línea `import './index.css';` porque ya no usaremos estilos personalizados.

Archivo completo (MODIFICADO):

```
// [MODIFICADO] - se eliminó import './index.css';

import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

3 Eliminar archivos CSS generados por Vite

Elimina completamente:

```
src/index.css
src/App.css
```

4 Modificar App.jsx para mostrar el título inicial

Ruta:

biblio-react/src/App.jsx

Descripción:

Creamos un título amarillo, centrado horizontalmente, con icono de Bootstrap (`bi-book-half`). Sin estilos personalizados, solo clases de Bootstrap.

Archivo completo (MODIFICADO):

```
// [MODIFICADO]

function App() {
  return (
    <main className="container pt-4">
      <div className="col-12 col-md-10 col-lg-8 mx-auto text-center">

        <h1 className="display-4 fw-bold text-warning mb-4">
          <i className="bi bi-book-half me-2"></i>
          Sistema de Biblioteca Personal
        </h1>

        <p className="lead text-light">
          Frontend en React + Vite con Bootstrap en modo oscuro.
        </p>

      </div>
    </main>
  );
}

export default App;
```

5 Validar correctamente

1. Si ya tienes el servidor corriendo, no ejecutes nada.
2. Si no está corriendo, ejecuta:

```
npm run dev
```

3. Ve a:

```
http://localhost:5173/
```

4. Debes ver:
 - o Modo oscuro activo.

- Título amarillo centrado horizontalmente.
- Icono de libro antes del título.
- Nada centrado verticalmente (correcto).
- Bootstrap funcionando bien.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 3).**

Dijiste:

listo vamos fase 3

ChatGPT dijo:

✅ FASE 3 — CREAR CARPETA `api` Y ARCHIVO `libros.js` (solo URL base + axios)

Objetivo de la fase:

Centralizar la comunicación con la API creando la carpeta `api` y el archivo `libros.js`, donde definiremos **axios** y la constante `urlBase`.

Aún **sin funciones**, solo estructura base.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
    └─ api\  
        └─ libros.js      (NUEVO)  
    └─ App.jsx            (EXISTENTE)  
    └─ main.jsx           (EXISTENTE)
```

En esta fase solamente se crea la carpeta `api` y el archivo `libros.js`.

1 Crear carpeta `api`

En la ruta:

`biblio-react/src/`

Crear carpeta:

`api`

2 Crear archivo libros.js

Ruta final:

biblio-react/src/api/libros.js

Descripción:

Aquí definimos:

- La constante `urlBase`
- La instancia de `axios` configurada
- Todavía **no agregamos funciones** como listar, crear, editar o eliminar

Archivo completo libros.js (NUEVO)

```
import axios from "axios";

const urlBase = "http://localhost:8080/api/libros";

const api = axios.create({
  baseURL: urlBase,
});

export { api, urlBase };
```

No hay pasos adicionales en esta fase.

Todo quedó listo para comenzar a programar la lógica real en la siguiente fase.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 4).**

Dijiste:

listo vamos a la fase 4

ChatGPT dijo:

FASE 4 — AGREGAR `listarLibros()` (solo lógica, sin pruebas aún)

Objetivo de la fase:

Agregar al archivo `libros.js` la función `listarLibros()` que realice la petición GET a la API. En esta fase solo se escribe la lógica; **no la probaremos todavía.**

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
├── src\  
│   └── api\  
│       └── libros.js      (MODIFICADO)
```

📄 Archivo completo libros.js (MODIFICADO)

Solo se marcan como [MODIFICADO] las líneas agregadas.
Todo lo previo se conserva tal cual.

```
import axios from "axios";  
  
const urlBase = "http://localhost:8080/api/libros";  
  
const api = axios.create({  
  baseURL: urlBase,  
});  
  
// [NUEVO]  
async function listarLibros() {  
  try {  
    const respuesta = await api.get("/");  
    return respuesta.data;  
  } catch (error) {  
    console.error("Error al listar libros:", error);  
    throw error;  
  }  
}  
  
export { api, urlBase, listarLibros };
```

👍 Con esto termina la lógica inicial del método GET.

No se prueba aún — eso vendrá cuando conectemos la tabla en la fase correspondiente.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 5).**

Dijiste:

listo vamos a la fase 5

ChatGPT dijo:

✓ FASE 5 — CREAR ListadoLibros.jsx (estructura básica)

Objetivo de la fase:

Crear el componente **ListadoLibros.jsx** con estructura mínima:

- Título "**Listado de Libros**", respetando el icono ya utilizado.
- Un pequeño texto introductorio.
- Conexión con **App.jsx** para mostrar este componente.
Aún **sin lógica**, sin tabla, sin botones.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
    └─ libros\  
        └─ ListadoLibros.jsx      (NUEVO)  
    └─ App.jsx                  (MODIFICADO)  
    └─ api\  
        └─ libros.js            (EXISTENTE)
```

1 Crear carpeta libros

En la ruta:

```
biblio-react/src/
```

Crear carpeta:

```
libros
```

2 Crear archivo ListadoLibros.jsx

Ruta:

```
biblio-react/src/libros/ListadoLibros.jsx
```

Descripción:

- Estructura básica del componente.
- Icono **igual** al que usamos en el título principal (`bi-book-half`).
- Título centrado horizontalmente.
- No lleva lógica ni efectos.

- No hay tabla todavía.

Archivo completo ListadoLibros.jsx (NUEVO)

```
function ListadoLibros() {
  return (
    <main className="container pt-4">
      <div className="col-12 col-md-10 col-lg-8 mx-auto">

        <h2 className="text-warning text-center mb-4">
          <i className="bi bi-book-half me-2"></i>
          Listado de Libros
        </h2>

        <p className="text-light text-center">
          Aquí podrás visualizar todos los libros registrados en el sistema.
        </p>

      </div>
    </main>
  );
}

export default ListadoLibros;
```

3 Modificar App.jsx para mostrar ListadoLibros.jsx

Descripción:

Por ahora no tenemos react-router-dom (eso llegará en Fase 7).
Así que simplemente importamos y mostramos el componente.

Ruta:

biblio-react/src/App.jsx

Archivo completo App.jsx (MODIFICADO)

```
// [MODIFICADO]
import ListadoLibros from "../libros/ListadoLibros.jsx";

function App() {
  return (
    <main className="container pt-4">
      <div className="col-12 col-md-10 col-lg-8 mx-auto text-center">

        <h1 className="display-4 fw-bold text-warning mb-4">
          <i className="bi bi-book-half me-2"></i>

```

```
    Sistema de Biblioteca Personal
  </h1>

  <p className="lead text-light mb-5">
    Frontend en React + Vite con Bootstrap en modo oscuro.
  </p>

  { /* [NUEVO] Mostrar el componente del Listado */ }
  <ListadoLibros />

</div>
</main>
);
}

export default App;
```

Validar

1. Ejecuta (si no está corriendo):

```
npm run dev
```

2. Ve a:

```
http://localhost:5173/
```

Debes ver:

- El título general del sistema.
 - El subtítulo con descripción.
 - Debajo, el nuevo título “**Listado de Libros**” con el icono.
 - Todo en modo oscuro y correctamente centrado de forma horizontal.
-

¿Continuamos? Responde: **listo para la siguiente fase (Fase 6).**

Dijiste:

ya realice los cambios, pero vamos a limpiar el archivo App.jsx, va a quedar de esta manera mas limpio: // [MODIFICADO] import ListadoLibros from "../libros/ListadoLibros.jsx"; function App() { return (<main className="container pt-4"> { /* [NUEVO] Mostrar el componente del Listado */ } <ListadoLibros /> </main>); } export default App;

ChatGPT dijo:

Perfecto, dejamos **App.jsx** exactamente como indicas, limpio y minimalista. Esta corrección **sigue siendo parte de la Fase 5**, por lo que **solo modificamos el archivo correspondiente**, sin avanzar de fase.

Aquí tienes el archivo completo.

App.jsx (MODIFICADO)

Ruta:

```
biblio-react/src/App.jsx
// [MODIFICADO]
import ListadoLibros from "../libros/ListadoLibros.jsx";

function App() {
  return (
    <main className="container pt-4">

      {/* [NUEVO] Mostrar el componente del Listado */}
      <ListadoLibros />

    </main>
  );
}

export default App;
```

Todo correcto, quedó mucho más claro y limpio. Cuando estés lista:

¿Continuamos? Responde: listo para la siguiente fase (Fase 6).

Dijiste:

listo vamos a la fase 6

ChatGPT dijo:

FASE 6 — TABLA BOOTSTRAP CON DATOS REALES (GET)

Objetivo de la fase:

Conectar `listarLibros()` al componente **ListadoLibros.jsx** usando **useEffect**, mostrar los libros en una **tabla Bootstrap** con encabezado azul (`table-primary`), y sin botones todavía.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
   └─ libros\  
       └─ ListadoLibros.jsx      (MODIFICADO)  
   └─ api\  
       └─ libros.js             (EXISTENTE)  
   └─ App.jsx                   (EXISTENTE)
```



1 Modificar ListadoLibros.jsx para consumir la API

Ruta:

biblio-react/src/libros/ListadoLibros.jsx

Descripción:

- Importar `useState`, `useEffect` y `listarLibros()`.
 - Llamar a la API al montar el componente.
 - Dibujar tabla Bootstrap con columnas: **ID, Título, Autor, Rating**.
 - Encabezado azul: `className="table-primary"` (solo Bootstrap).
-



Archivo completo ListadoLibros.jsx (MODIFICADO)

```
// [MODIFICADO]  
import { useEffect, useState } from "react";  
import { listarLibros } from "../api/libros.js";  
  
function ListadoLibros() {  
  
  // [NUEVO] Estado para los libros  
  const [libros, setLibros] = useState([]);  
  
  // [NUEVO] Cargar libros al iniciar  
  useEffect(() => {  
    async function cargarDatos() {  
      try {  
        const data = await listarLibros();  
        setLibros(data);  
      } catch (error) {  
        alert("Error al cargar los libros");  
      }  
    }  
    cargarDatos();  
  }, []);
```

```

return (
  <div className="col-12 col-md-10 col-lg-8 mx-auto">

    <h2 className="text-warning text-center mb-4">
      <i className="bi bi-book-half me-2"></i>
      Listado de Libros
    </h2>

    <p className="text-light text-center">
      Aquí podrás visualizar todos los libros registrados en el sistema.
    </p>

    { /* [NUEVO] Tabla con datos reales */ }
    <div className="table-responsive">
      <table className="table table-dark table-striped table-hover">
        <thead className="table-primary">
          <tr>
            <th>ID</th>
            <th>Título</th>
            <th>Autor</th>
            <th>Rating</th>
          </tr>
        </thead>
        <tbody>
          {libros.map((libro) => (
            <tr key={libro.id}>
              <td>{libro.id}</td>
              <td>{libro.titulo}</td>
              <td>{libro.autor}</td>
              <td>{libro.rating}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>

  </div>
);
}

export default ListadoLibros;

```



2 Validar implementación

1. Asegúrate de que el backend FastAPI esté corriendo en:
`http://localhost:8080/api/libros`
2. Si la app no está corriendo:

`npm run dev`

3. Entra a:

`http://localhost:5173/`

Debes ver:

- Título "**Listado de Libros**" con icono.
 - Tabla Bootstrap oscura.
 - Encabezado azul (`table-primary`).
 - Libros reales cargados desde la API.
 - **Sin botones aún** (eso llegará en una fase posterior).
-

¿Continuamos? Responde: **listo para la siguiente fase (Fase 7)**.

Dijiste:

listo vamos a la fase 7

ChatGPT dijo:

✅ FASE 7 — INSTALAR react-router-dom + CREAR Navegacion.jsx

Objetivo de la fase:

Agregar navegación con **react-router-dom**, crear un **Navbar Bootstrap** y definir la ruta inicial:

- `/` → **ListadoLibros**
(Las rutas `/agregar` y `/editar/:id` se agregarán **SOLO** en las fases correspondientes.)
-

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
├── src\  
│   ├── componentes\  
│   │   ├── Navegacion.jsx      (NUEVO)  
│   │   ├── App.jsx             (MODIFICADO)  
│   │   ├── main.jsx            (MODIFICADO)  
│   │   └── libros\  
│   │       └── ListadoLibros.jsx (EXISTENTE)
```

1 Instalar react-router-dom

Ejecuta en la terminal:

```
npm install react-router-dom
```

2 Crear el componente Navegacion.jsx

Ruta:

biblio-react/src/componentes/Navegacion.jsx

Descripción:

- Navbar oscuro de Bootstrap.
- Elemento "Biblioteca Personal" enlaza a /
- No agregar aún los enlaces a **/agregar** y **/editar/:id** — se agregan en fases posteriores.
- Sin estilos personalizados.

Archivo completo Navegacion.jsx (NUEVO)

```
import { Link } from "react-router-dom";

function Navegacion() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark border-bottom border-secondary mb-4">
      <div className="container">

        <Link className="navbar-brand fw-bold text-warning" to="/">
          <i className="bi bi-book-half me-2"></i>
          Biblioteca Personal
        </Link>

        <button
          className="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarNav"
        >
          <span className="navbar-toggler-icon"></span>
        </button>

        <div className="collapse navbar-collapse" id="navbarNav">
          <ul className="navbar-nav ms-auto">

            { /* Las rutas /agregar y /editar se agregarán en fases posteriores */ }

          </ul>
        </div>

      </div>
    </nav>
  );
}
```

```
export default Navegacion;
```

3 Modificar main.jsx para activar react-router-dom

Ruta:

biblio-react/src/main.jsx

Archivo completo main.jsx (MODIFICADO)

```
// [MODIFICADO]
import React from 'react';
import ReactDOM from 'react-dom/client';

import { BrowserRouter } from "react-router-dom"; // [NUEVO]

import App from './App.jsx';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    { /* [NUEVO] Envolver en router */ }
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

4 Modificar App.jsx para usar las rutas

Ruta:

biblio-react/src/App.jsx

Descripción:

- Agregar Navbar.
- Definir rutas con <Routes> y <Route>.
- Solo una ruta en esta fase: / → **ListadoLibros**

Archivo completo App.jsx (MODIFICADO)

```
// [MODIFICADO]
import { Routes, Route } from "react-router-dom";
```

```
import Navegacion from "../componentes/Navegacion.jsx";
import ListadoLibros from "../libros/ListadoLibros.jsx";

function App() {
  return (
    <main className="container pt-4">

      {/* [NUEVO] Navbar */}
      <Navegacion />

      {/* [NUEVO] Rutas */}
      <Routes>
        <Route path="/" element={<ListadoLibros />} />
      </Routes>

    </main>
  );
}

export default App;
```

5 Validar funcionamiento

1. Ejecuta:

```
npm run dev
```

2. Entra a:

```
http://localhost:5173/
```

3. Debes ver:

- Navbar Bootstrap.
- El título **Biblioteca Personal** en color amarillo dentro del Navbar.
- El listado de libros debajo.
- Navegación funcionando correctamente (aunque solo hay una ruta por ahora).

¿Continuamos? Responde: **listo para la siguiente fase (Fase 8).**

Dijiste:

listo vamos a la fase 8

ChatGPT dijo:

✅ **FASE 8 — CREAR AgregarLibro.jsx (formulario vacío, sin lógica)**

Objetivo de la fase:

Crear el componente **AgregarLibro.jsx** con los tres inputs (título, autor, rating), botones **Guardar** y **Cancelar**, agregar su ruta **/agregar** en App.jsx y el enlace correspondiente en el **Navbar**.

Sin lógica aún (no se conecta con la API en esta fase).

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
    └─ libros\  
        └─ AgregarLibro.jsx          (NUEVO)  
    └─ componentes\  
        └─ Navegacion.jsx          (MODIFICADO)  
    └─ App.jsx                      (MODIFICADO)  
    └─ libros\  
        └─ ListadoLibros.jsx        (EXISTENTE)
```

1 Crear archivo AgregarLibro.jsx**Ruta:**

biblio-react/src/libros/AgregarLibro.jsx

Descripción:

- Formulario sin lógica.
- Tres inputs: título, autor, rating (number).
- Botones: Guardar (primario), Cancelar (secundario).
- Diseño Bootstrap oscuro.

📄 Archivo completo AgregarLibro.jsx (NUEVO)

```
import { Link } from "react-router-dom";  
  
function AgregarLibro() {  
  return (  
    <div className="col-12 col-md-8 col-lg-6 mx-auto">  
  
      <h2 className="text-warning text-center mb-4">  
        <i className="bi bi-book-half me-2"></i>  
        Agregar Libro  
      </h2>  
  
      <form>
```

```

    <div className="mb-3">
      <label className="form-label text-light">Título</label>
      <input type="text" className="form-control" />
    </div>

    <div className="mb-3">
      <label className="form-label text-light">Autor</label>
      <input type="text" className="form-control" />
    </div>

    <div className="mb-3">
      <label className="form-label text-light">Rating (1-5)</label>
      <input type="number" className="form-control" min="1" max="5" />
    </div>

    <div className="d-flex gap-2">
      <button type="submit" className="btn btn-primary">
        Guardar
      </button>

      <Link to="/" className="btn btn-secondary">
        Cancelar
      </Link>
    </div>

  </form>

</div>
);
}

export default AgregarLibro;

```

2 Modificar Navegacion.jsx para agregar el enlace /agregar

Ruta:

biblio-react/src/componentes/Navegacion.jsx

Descripción:

Agregamos una opción "Agregar" en el menú.

Sin iconos todavía (se agregan hasta fases exactas si aplica).

Esta fase solo agrega la ruta.

Archivo completo Navegacion.jsx (MODIFICADO)

```
import { Link } from "react-router-dom";
```

```

function Navegacion() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark border-bottom border-
secondary mb-4">
      <div className="container">

        <Link className="navbar-brand fw-bold text-warning" to="/">
          <i className="bi bi-book-half me-2"></i>
            Biblioteca Personal
        </Link>

        <button
          className="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarNav"
        >
          <span className="navbar-toggler-icon"></span>
        </button>

        <div className="collapse navbar-collapse" id="navbarNav">
          <ul className="navbar-nav ms-auto">

            { /* [NUEVO] Enlace hacia /agregar */ }
            <li className="nav-item">
              <Link className="nav-link" to="/agregar">
                Agregar
              </Link>
            </li>

          </ul>
        </div>

      </div>
    </nav>
  );
}

export default Navegacion;

```

3 Modificar App.jsx para agregar la ruta /agregar

Ruta:

biblio-react/src/App.jsx

Archivo completo App.jsx (MODIFICADO)

```

// [MODIFICADO]
import { Routes, Route } from "react-router-dom";

```

```
import Navegacion from "../componentes/Navegacion.jsx";
import ListadoLibros from "../libros/ListadoLibros.jsx";
import AgregarLibro from "../libros/AgregarLibro.jsx"; // [NUEVO]

function App() {
  return (
    <main className="container pt-4">

      <Navegacion />

      <Routes>
        <Route path="/" element={<ListadoLibros />} />
        <Route path="/agregar" element={<AgregarLibro />} /> { /* [NUEVO] */ }
      </Routes>

    </main>
  );
}

export default App;
```

Validar

1. Ejecuta:

```
npm run dev
```

2. En el navegador:

- Ve a /
- Luego haz clic en **Agregar** en el menú.
- Debe mostrarse el formulario vacío.
- Botón Cancelar debe regresar al listado.

Todo sin lógica aún (eso llega en Fase 10).

¿Continuamos? Responde: **listo para la siguiente fase (Fase 9).**

Dijiste:

listo vamos a la fase 9

ChatGPT dijo:

 **FASE 9 — PROGRAMAR crearLibro() EN libros.js (POST + validaciones básicas)**

Objetivo de la fase:

Agregar al archivo `libros.js` la función **crearLibro()**, encargada de enviar un POST a la API. Incluye validaciones básicas antes de llamar al backend.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
   └─ api\  
      └─ libros.js          (MODIFICADO)
```

1 Modificar libros.js para agregar crearLibro()**Ruta:**

`biblio-react/src/api/libros.js`

Descripción:

- Se implementa POST a `/`.
 - Validaciones:
 - título obligatorio
 - autor obligatorio
 - rating entre 1 y 5
 - En caso de error se lanza excepción.
-

📄 Archivo completo libros.js (MODIFICADO)

```
import axios from "axios";  
  
const urlBase = "http://localhost:8080/api/libros";  
  
const api = axios.create({  
  baseURL: urlBase,  
});  
  
// [EXISTENTE]  
async function listarLibros() {  
  try {  
    const respuesta = await api.get("/");  
    return respuesta.data;  
  } catch (error) {  
    console.error("Error al listar libros:", error);  
    throw error;  
  }  
}
```



```

    }
  }

  // [NUEVO]
  async function crearLibro(libro) {
    // Validaciones básicas
    if (!libro.titulo || libro.titulo.trim() === "") {
      throw new Error("El título es obligatorio");
    }

    if (!libro.autor || libro.autor.trim() === "") {
      throw new Error("El autor es obligatorio");
    }

    const rating = Number(libro.rating);
    if (isNaN(rating) || rating < 1 || rating > 5) {
      throw new Error("El rating debe ser un número entre 1 y 5");
    }

    // Petición POST
    try {
      const respuesta = await api.post("/", libro);
      return respuesta.data;
    } catch (error) {
      console.error("Error al crear libro:", error);
      throw error;
    }
  }

export { api, urlBase, listarLibros, crearLibro };

```

✓ Final de la fase

En esta etapa **solo** se agrega la función; no se conecta al formulario todavía. La conexión al formulario se hace en la siguiente fase.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 10).**

Dijiste:

listo vamos a la fase 10

ChatGPT dijo:

✓ FASE 10 — Conectar AgregarLibro.jsx con crearLibro()

Formulario controlado + Submit + Redirección + Alertas

Objetivo de la fase:

Agregar toda la lógica al formulario para crear un libro:

- **useState** para controlar los campos.
- **manejarSubmit()** para enviar datos con `crearLibro()`.
- **navigate("/")** al guardar.
- Alertas básicas de éxito y error.

NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\
├── src\
│   ├── libros\
│   │   └── AgregarLibro.jsx      (MODIFICADO)
│   ├── api\
│   │   └── libros.js             (EXISTENTE)
│   └── App.jsx                   (EXISTENTE)
```



Modificar AgregarLibro.jsx para agregar toda la lógica

Ruta:

biblio-react/src/libros/AgregarLibro.jsx

Descripción:

- Tres estados: `titulo`, `autor`, `rating`.
- Submit que llama `crearLibro()`.
- Validaciones manejadas desde el backend y desde `crearLibro()`.
- Redirección tras guardar.



Archivo completo AgregarLibro.jsx (MODIFICADO)

```
// [MODIFICADO]
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import { crearLibro } from "../api/libros.js";

function AgregarLibro() {

  // [NUEVO] Estados del formulario
  const [titulo, setTitulo] = useState("");
  const [autor, setAutor] = useState("");
```

```
const [rating, setRating] = useState("");

const navigate = useNavigate(); // [NUEVO]

// [NUEVO] manejar envío del formulario
async function manejarSubmit(e) {
  e.preventDefault();

  try {
    const nuevoLibro = {
      titulo,
      autor,
      rating,
    };

    await crearLibro(nuevoLibro);

    alert("Libro creado correctamente");

    navigate("/"); // Redirigir al listado
  } catch (error) {
    alert(error.message || "Error al crear libro");
  }
}

return (
  <div className="col-12 col-md-8 col-lg-6 mx-auto">

    <h2 className="text-warning text-center mb-4">
      <i className="bi bi-book-half me-2"></i>
      Agregar Libro
    </h2>

    <form onSubmit={manejarSubmit}>
      <div className="mb-3">
        <label className="form-label text-light">Título</label>
        <input
          type="text"
          className="form-control"
          value={titulo}
          onChange={(e) => setTitulo(e.target.value)}
        />
      </div>

      <div className="mb-3">
        <label className="form-label text-light">Autor</label>
        <input
          type="text"
          className="form-control"
          value={autor}
          onChange={(e) => setAutor(e.target.value)}
        />
      </div>

      <div className="mb-3">
        <label className="form-label text-light">Rating (1-5)</label>
        <input
```

```

        type="number"
        className="form-control"
        min="1"
        max="5"
        value={rating}
        onChange={ (e) => setRating(e.target.value) }
      />
    </div>

    <div className="d-flex gap-2">
      <button
        type="submit"
        className="btn btn-primary"
      >
        Guardar
      </button>

      <Link to="/" className="btn btn-secondary">
        Cancelar
      </Link>
    </div>
  </form>
</div>
);
}

export default AgregarLibro;

```

✓ Validar

1. Entra a:

<http://localhost:5173/agregar>

2. Llena formulario con valores válidos.

3. Debe ocurrir:

- Mostrar alerta: **"Libro creado correctamente"**
- Redirigir a /
- Ver el nuevo libro en el listado

Si falla algo:

- Se mostrará `alert(error.message)`
- No redirige

¿Continuamos? Responde: **listo para la siguiente fase (Fase 11).**

Dijiste:

listo vamos a la fase 11

ChatGPT dijo:

✅ FASE 11 — obtenerLibroPorId() en libros.js

Objetivo de la fase:

Crear la función **obtenerLibroPorId()** para consultar un libro específico por su ID mediante un GET a la API.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
├── src\  
│   └── api\  
│       └── libros.js          (MODIFICADO)
```



Modificar libros.js para agregar obtenerLibroPorId()

Ruta:

biblio-react/src/api/libros.js

Descripción:

- Implementa un GET: `GET /{id}`
- Devuelve el objeto del libro.
- Si no existe, lanza error.



Archivo completo libros.js (MODIFICADO)

```
import axios from "axios";  
  
const urlBase = "http://localhost:8080/api/libros";  
  
const api = axios.create({  
  baseURL: urlBase,  
});  
  
// [EXISTENTE]  
async function listarLibros() {
```

```

    try {
      const respuesta = await api.get("/");
      return respuesta.data;
    } catch (error) {
      console.error("Error al listar libros:", error);
      throw error;
    }
  }

// [EXISTENTE]
async function crearLibro(libro) {
  if (!libro.titulo || libro.titulo.trim() === "") {
    throw new Error("El título es obligatorio");
  }

  if (!libro.autor || libro.autor.trim() === "") {
    throw new Error("El autor es obligatorio");
  }

  const rating = Number(libro.rating);
  if (isNaN(rating) || rating < 1 || rating > 5) {
    throw new Error("El rating debe ser un número entre 1 y 5");
  }

  try {
    const respuesta = await api.post("/", libro);
    return respuesta.data;
  } catch (error) {
    console.error("Error al crear libro:", error);
    throw error;
  }
}

// [NUEVO]
async function obtenerLibroPorId(id) {
  try {
    const respuesta = await api.get(`/ ${id}`);
    return respuesta.data;
  } catch (error) {
    console.error(`Error al obtener libro con id ${id}:`, error);
    throw new Error("No se pudo obtener el libro solicitado");
  }
}

export { api, urlBase, listarLibros, crearLibro, obtenerLibroPorId };

```

✓ ¿Qué sigue?

En la siguiente fase cargaremos la información del libro dentro del formulario de edición.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 12).**

Dijiste:

listo vamos a la fase 12

ChatGPT dijo:

✅ FASE 12 — Agregar botón de Editar + Ruta /editar/:id

Objetivo de la fase:

Agregar en el listado un **botón de Editar** por cada libro, con icono Bootstrap azul, y configurar la nueva ruta:

/editar/:id

Aún no se crea la pantalla de edición (eso corresponde a la Fase 13).
Aquí solo se agrega el botón + navegación.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
    └─ libros\  
        └─ ListadoLibros.jsx          (MODIFICADO)  
    └─ componentes\  
        └─ Navegacion.jsx            (MODIFICADO)  
    └─ App.jsx                        (MODIFICADO)  
    └─ libros\  
        └─ EditarLibro.jsx            (NUEVO - estructura vacía en Fase 13)
```

Nota: En esta fase NO se crea todavía EditarLibro.jsx.
Se creará en la próxima fase.
En esta fase solo preparamos la ruta.

1 Modificar ListadoLibros.jsx para agregar el botón Editar

Ruta:

biblio-react/src/libros/ListadoLibros.jsx

Descripción:

- Agregamos una columna nueva "Acciones".
- Botón con icono azul: `<i className="bi bi-pencil-square"></i>`
- Enlace con `Link` hacia `/editar/{id}`.



Archivo completo ListadoLibros.jsx (MODIFICADO)

```
// [MODIFICADO]
import { useEffect, useState } from "react";
import { Link } from "react-router-dom"; // [NUEVO]
import { listarLibros } from "../api/libros.js";

function ListadoLibros() {

  const [libros, setLibros] = useState([]);

  useEffect(() => {
    async function cargarDatos() {
      try {
        const data = await listarLibros();
        setLibros(data);
      } catch (error) {
        alert("Error al cargar los libros");
      }
    }
    cargarDatos();
  }, []);

  return (
    <div className="col-12 col-md-10 col-lg-8 mx-auto">

      <h2 className="text-warning text-center mb-4">
        <i className="bi bi-book-half me-2"></i>
        Listado de Libros
      </h2>

      <p className="text-light text-center">
        Aquí podrás visualizar todos los libros registrados en el sistema.
      </p>

      <div className="table-responsive">
        <table className="table table-dark table-striped table-hover">
          <thead className="table-primary">
            <tr>
              <th>ID</th>
              <th>Título</th>
              <th>Autor</th>
              <th>Rating</th>
              <th>Acciones</th> { /* [NUEVO] */ }
            </tr>
          </thead>
          <tbody>
            {libros.map((libro) => (
```



```

        <tr key={libro.id}>
          <td>{libro.id}</td>
          <td>{libro.titulo}</td>
          <td>{libro.autor}</td>
          <td>{libro.rating}</td>

          { /* [NUEVO] Botón Editar */ }
          <td>
            <Link
              to={`/editar/${libro.id}`}
              className="btn btn-sm btn-outline-primary"
            >
              <i className="bi bi-pencil-square me-1"></i>
              Editar
            </Link>
          </td>
        </tr>
      )}
    </tbody>
  </table>
</div>

</div>
);
}

export default ListadoLibros;

```

2 Modificar App.jsx para agregar la ruta `/editar/:id`

Ruta:

biblio-react/src/App.jsx

Archivo completo App.jsx (MODIFICADO)

```

// [MODIFICADO]
import { Routes, Route } from "react-router-dom";

import Navegacion from "../componentes/Navegacion.jsx";
import ListadoLibros from "../libros/ListadoLibros.jsx";
import AgregarLibro from "../libros/AgregarLibro.jsx";
import EditarLibro from "../libros/EditarLibro.jsx"; // [NUEVO]

function App() {
  return (
    <main className="container pt-4">

      <Navegacion />

```

```

<Routes>
  <Route path="/" element={<ListadoLibros />} />
  <Route path="/agregar" element={<AgregarLibro />} />
  <Route path="/editar/:id" element={<EditarLibro />} /> { /* [NUEVO] */}
</Routes>

</main>
);
}

export default App;

```

3 Modificar Navegacion.jsx (Aún NO agregamos enlace a Editar)

El menú **NO debe tener** botón de editar (solo va en cada fila).
Pero sí ajustamos comentarios.

Ruta:

biblio-react/src/componentes/Navegacion.jsx



Archivo completo Navegacion.jsx (MODIFICADO)

```

import { Link } from "react-router-dom";

function Navegacion() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark border-bottom border-secondary mb-4">
      <div className="container">

        <Link className="navbar-brand fw-bold text-warning" to="/">
          <i className="bi bi-book-half me-2"></i>
          Biblioteca Personal
        </Link>

        <button
          className="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarNav"
        >
          <span className="navbar-toggler-icon"></span>
        </button>

        <div className="collapse navbar-collapse" id="navbarNav">

```

```

<ul className="navbar-nav ms-auto">

  {/* Ruta agregar ya estaba */}
  <li className="nav-item">
    <Link className="nav-link" to="/agregar">
      Agregar
    </Link>
  </li>

  {/* No agregar enlace a Editar aquí */}

</ul>
</div>

</div>
</nav>
);
}

export default Navegacion;

```

✓ Validar

1. Ir al navegador:

`http://localhost:5173/`

2. Verificar que en la tabla aparece ahora la columna **Acciones**.
3. Cada fila debe mostrar:

[icono lápiz azul] Editar

4. Al dar clic en Editar:
→ Debe navegar a:

`/editar/ID_DEL_LIBRO`

5. Como la siguiente fase aún no crea el formulario de edición, verás un error (pantalla en blanco o componente inexistente).
Esto es **correcto** para esta etapa.
-

¿Continuamos? Responde: **listo para la siguiente fase (Fase 13)**.

Dijiste:

listo vamos a la fase 13

ChatGPT dijo:

✅ FASE 13 — CREAR EditarLibro.jsx (estructura vacía, igual que AgregarLibro)

Objetivo de la fase:

Crear el archivo **EditarLibro.jsx** con la **misma estructura visual** que **AgregarLibro.jsx** (mismos inputs y botones), pero **sin lógica todavía**.

La precarga de datos y conexión con la API se hará en las fases 14 y 16.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
    └─ libros\  
        └─ EditarLibro.jsx      (NUEVO)  
    └─ libros\  
        └─ ListadoLibros.jsx    (EXISTENTE)  
    └─ App.jsx                  (EXISTENTE)  
    └─ api\  
        └─ libros.js            (EXISTENTE)
```

1 Crear archivo EditarLibro.jsx

Ruta:

biblio-react/src/libros/EditarLibro.jsx

Descripción:

- Estructura idéntica al formulario de agregar.
- Inputs de título, autor y rating.
- Botones: **Guardar Cambios** y **Cancelar**.
- Sin lógica de precarga ni submit aún.
- Sin estados, sin useEffect, sin API.
- Todo eso llegará en Fases 14 y 16.

📄 Archivo completo EditarLibro.jsx (NUEVO)

```
import { Link } from "react-router-dom";  
  
function EditarLibro() {  
  return (  
    <div className="col-12 col-md-8 col-lg-6 mx-auto">
```

```

<h2 className="text-warning text-center mb-4">
  <i className="bi bi-book-half me-2"></i>
  Editar Libro
</h2>

<form>

  <div className="mb-3">
    <label className="form-label text-light">Título</label>
    <input type="text" className="form-control" />
  </div>

  <div className="mb-3">
    <label className="form-label text-light">Autor</label>
    <input type="text" className="form-control" />
  </div>

  <div className="mb-3">
    <label className="form-label text-light">Rating (1-5)</label>
    <input type="number" className="form-control" min="1" max="5" />
  </div>

  <div className="d-flex gap-2">
    <button type="submit" className="btn btn-primary">
      Guardar Cambios
    </button>

    <Link to="/" className="btn btn-secondary">
      Cancelar
    </Link>
  </div>

</form>

</div>
);
}

export default EditarLibro;

```

✓ Validar

1. Ir a:

<http://localhost:5173/editar/1>

2. Debes ver el formulario vacío.
3. Los campos **no** deben tener datos aún (eso viene en la Fase 14).
4. Botón Cancelar te debe regresar al inicio.
5. Guardar Cambios no debe hacer nada todavía (sin lógica).

Todo correcto para esta fase.

¿Continuamos? Responde: listo para la siguiente fase (Fase 14).

Dijiste:

listo vamos a la fase 14

ChatGPT dijo:

✅ FASE 14 — EditarLibro.jsx: Precargar datos

(useParams + obtenerLibroPorId + llenar el formulario)

Objetivo de la fase:

Hacer que el formulario de edición cargue automáticamente los datos del libro seleccionado:

- Obtener el **id** desde la URL con `useParams()`
 - Llamar a **obtenerLibroPorId()**
 - Llenar los inputs con los datos reales
 - Manejar estados para título, autor y rating
 - Aún **NO** guardamos cambios (eso será en Fase 16)
-

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
    └─ libros\  
        └─ EditarLibro.jsx      (MODIFICADO)  
    └─ api\  
        └─ libros.js           (EXISTENTE)  
    └─ App.jsx                 (EXISTENTE)
```

1 Modificar EditarLibro.jsx para precargar datos reales

Ruta:

biblio-react/src/libros/EditarLibro.jsx

Descripción:

- Agregamos useState, useEffect, useParams.
- Llamamos a obtenerLibroPorId(id).
- Rellenamos los inputs con los valores reales.
- NO se implementa aún la actualización (eso llega en Fase 16).



Archivo completo EditarLibro.jsx (MODIFICADO)

```
// [MODIFICADO]
import { useEffect, useState } from "react";
import { Link, useParams } from "react-router-dom";
import { obtenerLibroPorId } from "../api/libros.js";

function EditarLibro() {

  // [NUEVO] Obtener el id de la URL
  const { id } = useParams();

  // [NUEVO] Estados del formulario
  const [titulo, setTitulo] = useState("");
  const [autor, setAutor] = useState("");
  const [rating, setRating] = useState("");

  // [NUEVO] Cargar datos del libro al iniciar
  useEffect(() => {
    async function cargarLibro() {
      try {
        const libro = await obtenerLibroPorId(id);
        setTitulo(libro.titulo);
        setAutor(libro.autor);
        setRating(libro.rating);
      } catch (error) {
        alert("Error al cargar los datos del libro");
      }
    }
    cargarLibro();
  }, [id]);

  return (
    <div className="col-12 col-md-8 col-lg-6 mx-auto">

      <h2 className="text-warning text-center mb-4">
        <i className="bi bi-book-half me-2"></i>
        Editar Libro
      </h2>

      { /* Aún NO conectamos submit, eso es en Fase 16 */ }
      <form>

        <div className="mb-3">
          <label className="form-label text-light">Título</label>
          <input
            type="text"

```

```

        className="form-control"
        value={titulo}
        onChange={ (e) => setTitulo(e.target.value) }
      />
    </div>

    <div className="mb-3">
      <label className="form-label text-light">Autor</label>
      <input
        type="text"
        className="form-control"
        value={autor}
        onChange={ (e) => setAutor(e.target.value) }
      />
    </div>

    <div className="mb-3">
      <label className="form-label text-light">Rating (1-5)</label>
      <input
        type="number"
        className="form-control"
        min="1"
        max="5"
        value={rating}
        onChange={ (e) => setRating(e.target.value) }
      />
    </div>

    <div className="d-flex gap-2">
      <button type="submit" className="btn btn-primary">
        Guardar Cambios
      </button>

      <Link to="/" className="btn btn-secondary">
        Cancelar
      </Link>
    </div>

  </form>

</div>
);
}

export default EditarLibro;

```

✓ Validar funcionalidad

1. Ir a un libro desde el listado → Editar
Ejemplo:

<http://localhost:5173/editar/1>

2. Debes ver:

- Inputs llenos con los valores reales.
- El formulario editable.
- Botón Guardar Cambios sin acción (todavía).
- Cancelar te regresa al listado.

Todo correcto para esta fase.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 15).**

Dijiste:

listo vamos a la fase 15

ChatGPT dijo:

✅ **FASE 15 — Programar actualizarLibro() en libros.js**

(PUT por id)

Objetivo de la fase:

Agregar al archivo `libros.js` la función **actualizarLibro()**, encargada de enviar una petición **PUT** para modificar un libro existente según su ID.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
├── src\  
│   └── api\  
│       └── libros.js (MODIFICADO)
```

Modificar libros.js para agregar actualizarLibro()

Ruta:

`biblio-react/src/api/libros.js`

Descripción:

- Implementa `PUT /{id}`.
- Reutiliza las mismas validaciones básicas de creación.
- Lanza errores en caso de fallas.



Archivo completo libros.js (MODIFICADO)

```
import axios from "axios";

const urlBase = "http://localhost:8080/api/libros";

const api = axios.create({
  baseURL: urlBase,
});

// [EXISTENTE]
async function listarLibros() {
  try {
    const respuesta = await api.get("/");
    return respuesta.data;
  } catch (error) {
    console.error("Error al listar libros:", error);
    throw error;
  }
}

// [EXISTENTE]
async function crearLibro(libro) {
  if (!libro.titulo || libro.titulo.trim() === "") {
    throw new Error("El título es obligatorio");
  }

  if (!libro.autor || libro.autor.trim() === "") {
    throw new Error("El autor es obligatorio");
  }

  const rating = Number(libro.rating);
  if (isNaN(rating) || rating < 1 || rating > 5) {
    throw new Error("El rating debe ser un número entre 1 y 5");
  }

  try {
    const respuesta = await api.post("/", libro);
    return respuesta.data;
  } catch (error) {
    console.error("Error al crear libro:", error);
    throw error;
  }
}

// [EXISTENTE]
async function obtenerLibroPorId(id) {
  try {
```

```
    const respuesta = await api.get(`/${id}`);
    return respuesta.data;
  } catch (error) {
    console.error(`Error al obtener libro con id ${id}:`, error);
    throw new Error("No se pudo obtener el libro solicitado");
  }
}

// [NUEVO]
async function actualizarLibro(id, libro) {
  // Validaciones
  if (!libro.titulo || libro.titulo.trim() === "") {
    throw new Error("El título es obligatorio");
  }

  if (!libro.autor || libro.autor.trim() === "") {
    throw new Error("El autor es obligatorio");
  }

  const rating = Number(libro.rating);
  if (isNaN(rating) || rating < 1 || rating > 5) {
    throw new Error("El rating debe ser un número entre 1 y 5");
  }

  try {
    const respuesta = await api.put(`/${id}`, libro);
    return respuesta.data;
  } catch (error) {
    console.error(`Error al actualizar libro con id ${id}:`, error);
    throw new Error("No se pudo actualizar el libro");
  }
}

export {
  api,
  urlBase,
  listarLibros,
  crearLibro,
  obtenerLibroPorId,
  actualizarLibro
};
```

✓ Fin de la Fase 15

En esta fase solo se creó la función PUT.

En la próxima fase conectaremos esta función al formulario y agregaremos la lógica de **guardar cambios**.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 16)**.

Dijiste:

listo vamos a la fase 16

ChatGPT dijo:

✅ FASE 16 — EditarLibro.jsx: Conectar formulario a actualizarLibro()

(Guardar cambios + Redirección + Alertas)

Objetivo de la fase:

Completar el flujo de edición para que el formulario:

- Envíe datos al backend usando **actualizarLibro()**
- Muestre alertas de éxito/error
- Redirija al listado después de guardar
- Mantenga la precarga de datos ya implementada en la fase 14

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
    └─ libros\  
        └─ EditarLibro.jsx      (MODIFICADO)  
    └─ api\  
        └─ libros.js           (EXISTENTE)  
    └─ App.jsx                 (EXISTENTE)
```

1 Modificar EditarLibro.jsx para agregar toda la lógica de actualización

Ruta:

biblio-react/src/libros/EditarLibro.jsx

Descripción:

- Usar `actualizarLibro()` al enviar el formulario.
 - Manejar `useNavigate()` para regresar a /.
 - Mantener los estados y la precarga ya implementada.
 - Validaciones ocurren en `actualizarLibro()`.
-



Archivo completo EditarLibro.jsx (MODIFICADO)

```
// [MODIFICADO]
import { useEffect, useState } from "react";
import { Link, useNavigate, useParams } from "react-router-dom";
import { obtenerLibroPorId, actualizarLibro } from "../api/libros.js";

function EditarLibro() {

  const { id } = useParams();
  const navigate = useNavigate(); // [NUEVO]

  const [titulo, setTitulo] = useState("");
  const [autor, setAutor] = useState("");
  const [rating, setRating] = useState("");

  // [EXISTENTE] Precargar datos
  useEffect(() => {
    async function cargarLibro() {
      try {
        const libro = await obtenerLibroPorId(id);
        setTitulo(libro.titulo);
        setAutor(libro.autor);
        setRating(libro.rating);
      } catch (error) {
        alert("Error al cargar los datos del libro");
      }
    }
    cargarLibro();
  }, [id]);

  // [NUEVO] Guardar cambios
  async function manejarSubmit(e) {
    e.preventDefault();

    try {
      const libroActualizado = {
        titulo,
        autor,
        rating
      };

      await actualizarLibro(id, libroActualizado);

      alert("Libro actualizado correctamente");

      navigate("/"); // regresar al listado
    } catch (error) {
      alert(error.message || "Error al actualizar libro");
    }
  }

  return (
    <div className="col-12 col-md-8 col-lg-6 mx-auto">

      <h2 className="text-warning text-center mb-4">
```

```

    <i className="bi bi-book-half me-2"></i>
    Editar Libro
</h2>

<form onSubmit={manejarSubmit}>
  <div className="mb-3">
    <label className="form-label text-light">Título</label>
    <input
      type="text"
      className="form-control"
      value={titulo}
      onChange={(e) => setTitulo(e.target.value)}
    />
  </div>

  <div className="mb-3">
    <label className="form-label text-light">Autor</label>
    <input
      type="text"
      className="form-control"
      value={autor}
      onChange={(e) => setAutor(e.target.value)}
    />
  </div>

  <div className="mb-3">
    <label className="form-label text-light">Rating (1-5)</label>
    <input
      type="number"
      className="form-control"
      min="1"
      max="5"
      value={rating}
      onChange={(e) => setRating(e.target.value)}
    />
  </div>

  <div className="d-flex gap-2">
    <button type="submit" className="btn btn-primary">
      Guardar Cambios
    </button>

    <Link to="/" className="btn btn-secondary">
      Cancelar
    </Link>
  </div>

</form>

</div>
);
}

export default EditarLibro;

```

✓ Validar funcionamiento

1. Entra al listado.
2. Haz clic en **Editar** en cualquier libro.
3. Cambia valores y presiona **Guardar Cambios**.
4. Debe ocurrir:
 - Mostrar alerta “*Libro actualizado correctamente*”
 - Redirigir al listado
 - Ver los cambios reflejados inmediatamente (porque el GET se ejecuta en el listado)

Si algo falla, verás una alerta con el mensaje de error.

¿Continuamos? Responde: **listo para la siguiente fase (Fase 17)**.

Dijiste:

listo vamos a la fase 17

ChatGPT dijo:

✓ FASE 17 — `eliminarLibro()` en `libros.js`

(DELETE por id)

Objetivo de la fase:

Agregar al archivo `libros.js` la función `eliminarLibro()`, que enviará una petición DELETE al backend para borrar un libro por su ID.

Esta fase **solo crea la función**.

El botón de eliminación se agregará en la Fase 18.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\
├── src\
│   └── api\
│       └── libros.js          (MODIFICADO)
```

1 Modificar libros.js para incluir eliminarLibro()

Ruta:

biblio-react/src/api/libros.js

Descripción:

- Se implementa DELETE /{id}
- Manejo básico de errores
- Devuelve la respuesta del backend

Archivo completo libros.js (MODIFICADO)

```
import axios from "axios";

const urlBase = "http://localhost:8080/api/libros";

const api = axios.create({
  baseURL: urlBase,
});

// [EXISTENTE]
async function listarLibros() {
  try {
    const respuesta = await api.get("/");
    return respuesta.data;
  } catch (error) {
    console.error("Error al listar libros:", error);
    throw error;
  }
}

// [EXISTENTE]
async function crearLibro(libro) {
  if (!libro.titulo || libro.titulo.trim() === "") {
    throw new Error("El título es obligatorio");
  }

  if (!libro.autor || libro.autor.trim() === "") {
    throw new Error("El autor es obligatorio");
  }

  const rating = Number(libro.rating);
  if (isNaN(rating) || rating < 1 || rating > 5) {
    throw new Error("El rating debe ser un número entre 1 y 5");
  }
}
```



```
    try {
      const respuesta = await api.post("/", libro);
      return respuesta.data;
    } catch (error) {
      console.error("Error al crear libro:", error);
      throw error;
    }
  }

// [EXISTENTE]
async function obtenerLibroPorId(id) {
  try {
    const respuesta = await api.get(`/ ${id}`);
    return respuesta.data;
  } catch (error) {
    console.error(`Error al obtener libro con id ${id}:`, error);
    throw new Error("No se pudo obtener el libro solicitado");
  }
}

// [EXISTENTE]
async function actualizarLibro(id, libro) {
  if (!libro.titulo || libro.titulo.trim() === "") {
    throw new Error("El título es obligatorio");
  }

  if (!libro.autor || libro.autor.trim() === "") {
    throw new Error("El autor es obligatorio");
  }

  const rating = Number(libro.rating);
  if (isNaN(rating) || rating < 1 || rating > 5) {
    throw new Error("El rating debe ser un número entre 1 y 5");
  }

  try {
    const respuesta = await api.put(`/ ${id}`, libro);
    return respuesta.data;
  } catch (error) {
    console.error(`Error al actualizar libro con id ${id}:`, error);
    throw new Error("No se pudo actualizar el libro");
  }
}

// [NUEVO]
async function eliminarLibro(id) {
  try {
    const respuesta = await api.delete(`/ ${id}`);
    return respuesta.data;
  } catch (error) {
    console.error(`Error al eliminar libro con id ${id}:`, error);
    throw new Error("No se pudo eliminar el libro");
  }
}

export {
```

```
api,  
urlBase,  
listarLibros,  
crearLibro,  
obtenerLibroPorId,  
actualizarLibro,  
eliminarLibro    // [NUEVO]  
};
```

✓ Fin de la Fase 17

Ya tenemos la función DELETE lista.
En la siguiente fase agregaremos:

- Botón rojo **Eliminar**
 - Icono
 - Confirmación básica (`confirm()`)
 - Refrescar el listado después de eliminar
-

¿Continuamos? Responde: **listo para la siguiente fase (Fase 18).**

Dijiste:

listo vamos a la fase 18

ChatGPT dijo:

🎨 FASE 18 — Agregar botón de ELIMINAR en el listado

(Eliminar registro + Confirmación + Refrescar tabla)

Objetivo de la fase:

Agregar en cada fila del listado un **botón rojo de Eliminar**, con icono, que:

- Pida confirmación con `confirm()`
- Llame a `eliminarLibro(id)`
- Recargue la tabla después de eliminar
- Muestre alertas básicas

No se crea pantalla nueva. Todo ocurre desde el listado.

📁 NUEVOS ARCHIVOS / MODIFICACIONES

```
biblio-react\  
└─ src\  
   └─ libros\  
       └─ ListadoLibros.jsx      (MODIFICADO)  
   └─ api\  
       └─ libros.js             (EXISTENTE)  
   └─ App.jsx                   (EXISTENTE)
```

**1**

Modificar ListadoLibros.jsx para agregar funcionalidad de eliminación

Ruta:

biblio-react/src/libros/ListadoLibros.jsx

Descripción:

- Importar **eliminarLibro**
 - Crear función `borrar(id)`
 - Confirmar → eliminar → recargar listado
 - Agregar botón rojo con ícono Bootstrap `<i className="bi bi-trash"></i>`
-



Archivo completo ListadoLibros.jsx (MODIFICADO)

```
// [MODIFICADO]  
import { useEffect, useState } from "react";  
import { Link } from "react-router-dom";  
import { listarLibros, eliminarLibro } from "../api/libros.js"; // [NUEVO]  
  
function ListadoLibros() {  
  
  const [libros, setLibros] = useState([]);  
  
  // [EXISTENTE] Cargar libros  
  async function cargarDatos() {  
    try {  
      const data = await listarLibros();  
      setLibros(data);  
    } catch (error) {  
      alert("Error al cargar los libros");  
    }  
  }  
  
  useEffect(() => {  
    cargarDatos();  
  }, []);  
}
```

```
// [NUEVO] Eliminar libro
async function borrar(id) {
  const confirmar = confirm("¿Seguro que deseas eliminar este libro?");
  if (!confirmar) return;

  try {
    await eliminarLibro(id);
    alert("Libro eliminado correctamente");
    cargarDatos(); // refresca la tabla
  } catch (error) {
    alert(error.message || "Error al eliminar libro");
  }
}

return (
  <div className="col-12 col-md-10 col-lg-8 mx-auto">

    <h2 className="text-warning text-center mb-4">
      <i className="bi bi-book-half me-2"></i>
      Listado de Libros
    </h2>

    <p className="text-light text-center">
      Aquí podrás visualizar todos los libros registrados en el sistema.
    </p>

    <div className="table-responsive">
      <table className="table table-dark table-striped table-hover">
        <thead className="table-primary">
          <tr>
            <th>ID</th>
            <th>Título</th>
            <th>Autor</th>
            <th>Rating</th>
            <th>Acciones</th>
          </tr>
        </thead>
        <tbody>
          {libros.map((libro) => (
            <tr key={libro.id}>
              <td>{libro.id}</td>
              <td>{libro.titulo}</td>
              <td>{libro.autor}</td>
              <td>{libro.rating}</td>

              <td className="d-flex gap-2">

                {/* Botón Editar */}
                <Link
                  to={`/editar/${libro.id}`}
                  className="btn btn-sm btn-outline-primary"
                >
                  <i className="bi bi-pencil-square me-1"></i>
                  Editar
                </Link>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
)
```

```

        { /* [NUEVO] Botón Eliminar */ }
        <button
          onClick={() => borrar(libro.id)}
          className="btn btn-sm btn-outline-danger"
        >
          <i className="bi bi-trash me-1"></i>
          Eliminar
        </button>

      </td>

    </tr>
  )}
</tbody>
</table>
</div>

</div>
);
}

export default ListadoLibros;

```

✓ Validar el funcionamiento

1. Ve al listado:
2. `http://localhost:5173/`
3. Verifica que ahora aparecen dos botones en cada fila:
 -  Editar (azul)
 -  Eliminar (rojo)
4. Al hacer clic en **Eliminar**:
 - Debe mostrar `confirm("¿Seguro que deseas eliminar este libro?")`
 - Si confirmas, elimina el registro
 - Mostrar alerta: *"Libro eliminado correctamente"*
 - La tabla debe recargarse
5. Si el backend rechaza la operación, se mostrará un alert con el error.

PROYECTO COMPLETADO — CRUD COMPLETO DE BIBLIOTECA PERSONAL

Has construido un sistema **profesional** en **React + Vite**, con:

-  Navegación react-router-dom

- 📄 Listado con datos reales (GET)
- ➕ Crear libros (POST)
- ✏️ Editar libros (GET + PUT) con precarga
- 🗑️ Eliminar libros (DELETE)
- 🎨 Bootstrap 5 en modo oscuro completamente integrado
- 🔗 Axios centralizado
- ⚙️ Formularios controlados con estado
- 🔄 Refresco automático de datos
- 📁 Estructura ordenada para crecimiento del proyecto

Este flujo implementa el **CRUD completo** con buenas prácticas, navegación, componentes modulares y comunicación API REST.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)